

Gustavo Miguel Martins de Oliveira - RM 566666

Manuella Rinaldi - RM 567915

Mariana de Paula Aguiar - RM 566850

FIAP

Análise e Desenvolvimento de Sistemas - 1TDSPB

DataCare Solutions

Solução do Bem: uma plataforma voltada à ONG Turma do Bem para organizar dados e otimizar o atendimento odontológico.

São Paulo

2026

Objetivo e Escopo do Projeto

A Solução do Bem é um sistema de agendamento desenvolvido para a ONG Turma do Bem, que oferece atendimento odontológico gratuito a crianças, adolescentes em situação de vulnerabilidade social e mulheres que em decorrência de uma violência doméstica, tiveram sua saúde bucal afetada. A TDB atua em todo o Brasil e em mais de 12 países.

O sistema permite o cadastro e gerenciamento de beneficiários, profissionais voluntários, consultas odontológicas, locais de atendimento e instituições parceiras. A solução é exposta como uma API REST desenvolvida com Quarkus, permitindo comunicação direta com o front-end.

Limites de atuação:

- Cadastro, consulta, atualização e exclusão de beneficiários e profissionais
- Agendamento e gerenciamento de consultas odontológicas
- Cadastro de instituições e locais de atendimento
- Exposição de endpoints REST para integração com front-end

Principais Funcionalidades

1. Gestão de Beneficiários

- Cadastrar novo beneficiário com dados pessoais, responsável e programa
- Listar todos os beneficiários cadastrados
- Buscar beneficiário por CPF
- Atualizar dados de um beneficiário
- Excluir beneficiário do sistema

2. Gestão de Profissionais

- Cadastrar profissional voluntário com nome, CPF, CRO e especialização
- Listar todos os profissionais
- Buscar profissional por CPF (utilizado para autenticação no front-end)
- Atualizar e excluir profissional

3. Agendamento de Consultas

- Agendar consulta associando profissional, beneficiário e local
- Listar e buscar consultas por código
- Atualizar e cancelar consultas

4. Métodos com Lógica de Negócio

Foram implementados 4 métodos com regras de negócio relevantes:

Link do Repositório do GitHub:

<https://github.com/GustavoMiguelk/sprint-04-ddd.git>

Método 1 — Validação de Programa por Idade e Sexo

Localização: TurmaDoBem.java (case 3 do switch)

Regra: O programa 'Dentista do Bem' aceita apenas menores de 18 anos. O programa 'Apolônias do Bem' aceita apenas mulheres. Caso as regras sejam violadas, o cadastro é bloqueado com mensagem explicativa.

```
if (idPgm == 0 && idade >= 18) { mensagem("Somente menores de idade podem participar."); break; }  
if (idPgm == 1 && sexo.equals("masculino")) { mensagem("Somente mulheres podem participar."); break; }
```

Método 2 — Cadastro Automático de Responsável

Localização: TurmaDoBem.java (case 3 do switch)

Regra: Se o beneficiário for menor de 18 anos, o sistema solicita os dados do responsável. Caso contrário, o próprio beneficiário é registrado como responsável de si mesmo.

```
if (idade < 18) { resp = new Responsavel(texto("Nome:"),  
texto("CPF:"), texto("Contato:")); }  
else { resp = new Responsavel(nomeBen, cpfBen,  
texto("Contato:")); }
```

Método 3 — Geração de Código Único de Consulta (UUID)

Localização: TurmaDoBem.java (case 5 do switch)

Regra: Cada consulta recebe automaticamente um código único gerado via UUID, garantindo rastreabilidade e evitando duplicatas.

```
String codigo = UUID.randomUUID().toString();
```

Método 4 — Validação de Agenda do Profissional

Localização: TurmaDoBem.java (case 5 do switch) e Agenda.java

Regra: Antes de agendar uma consulta, o sistema verifica se o profissional já possui uma agenda criada. Se não tiver, cria automaticamente. A consulta é então adicionada à agenda do profissional.

```
if (prof.getAgenda() == null) prof.setAgenda(new Agenda(prof));  
prof.getAgenda().adicionarConsulta(consulta);
```

Tabela de Endpoints (API RESTful)

- Beneficiário

Recurso	Verbo HTTP	URI	Status	Descrição
Beneficiário	GET	/beneficiario	200 OK	Lista todos os beneficiários
Beneficiário	GET	/beneficiario/{cpf}	200 OK / 404	Busca beneficiário por CPF
Beneficiário	POST	/beneficiario	201 Created	Cadastra novo beneficiário
Beneficiário	PUT	/beneficiario/{cpf}	200 OK / 404	Atualiza dados do beneficiário
Beneficiário	DELETE	/beneficiario/{cpf}	200 OK / 404	Exclui beneficiário

- Profissional

Recurso	Verbo HTTP	URI	Status OK	Descrição
Profissional	GET	/profissional	200 OK	Lista todos os profissionais
Profissional	GET	/profissional/{cpf}	200 OK / 404	Busca profissional por CPF
Profissional	POST	/profissional	201 Created	Cadastra novo profissional
Profissional	PUT	/profissional/{cpf}	200 OK / 404	Atualiza dados do profissional
Profissional	DELETE	/profissional/{cpf}	200 OK / 404	Exclui profissional

- Consulta

Recurso	Verbo HTTP	URI	Status OK	Descrição
Consulta	GET	/consulta	200 OK	Lista todas as consultas
Consulta	GET	/consulta/{codigo}	200 OK / 404	Busca consulta por código
Consulta	POST	/consulta	201 Created	Agenda nova consulta
Consulta	PUT	/consulta/{codigo}	200 OK / 404	Atualiza consulta
Consulta	DELETE	/consulta/{codigo}	200 OK / 404	Cancela consulta

- Responsável

Recurso	Verbo HTTP	URI	Status OK	Descrição
Responsável	GET	/responsavel	200 OK	Lista todos os responsáveis
Responsável	GET	/responsavel/{cpf}	200 OK / 404	Busca responsável por CPF
Responsável	POST	/responsavel	201 Created	Cadastra novo responsável
Responsável	PUT	/responsavel/{cpf}	200 OK / 404	Atualiza responsável
Responsável	DELETE	/responsavel/{cpf}	200 OK / 404	Exclui responsável

Como rodar a aplicação

- Pré-requisitos

- Java JDK 17 ou superior
- IntelliJ IDEA (recomendado) ou outro IDE com suporte a Maven
- Conexão com a internet (banco Oracle FIAP)
- Maven (incluído no IntelliJ IDEA)

- Passo a Passo

1. Clone ou extraia o projeto na sua máquina.
2. Abra o IntelliJ IDEA e selecione: File → Open → pasta do projeto sprint-04-ddd.
3. Aguarde o IntelliJ indexar o projeto e baixar as dependências do Maven.
4. Verifique se o arquivo application.properties está configurado:

`quarkus.http.port=8080`

`quarkus.http.cors.origins=http://localhost:5173`

`quarkus.http.cors.methods=GET, POST, PUT, DELETE, OPTIONS`

`quarkus.http.cors.headers=Content-Type, Accept, Authorization`

5. No painel Maven (lado direito do IntelliJ), expanda:
sprint-04-ddd → Plugins → quarkus → clique duas vezes em quarkus:dev
6. Aguarde a mensagem: Listening on: http://localhost:8080
7. Acesse os endpoints:

`https://sprint-04-ddd.onrender.com/responsavel`

`https://sprint-04-ddd.onrender.com/beneficiario`

`https://sprint-04-ddd.onrender.com/consulta`

`https://sprint-04-ddd.onrender.com/local`

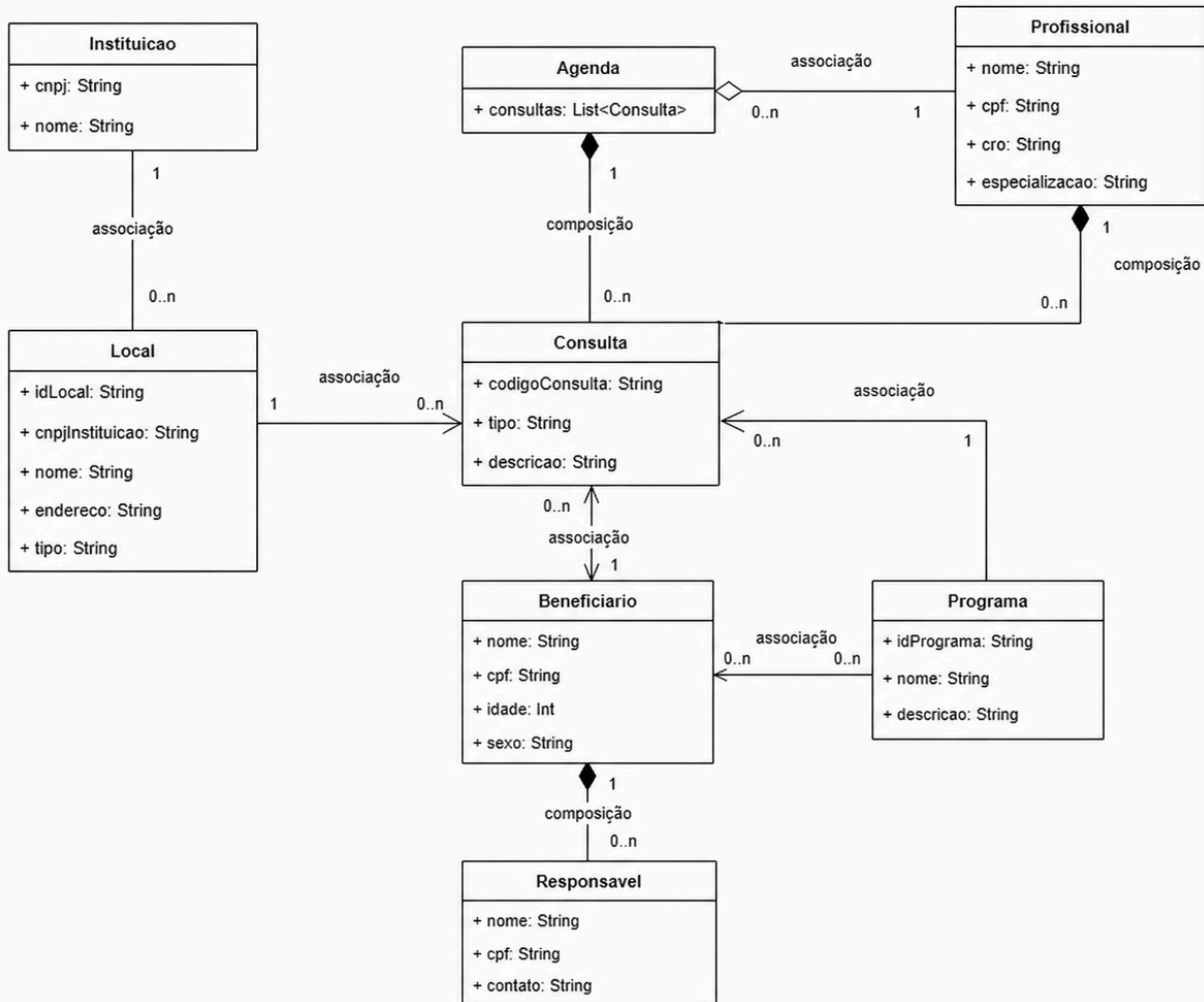
`https://sprint-04-ddd.onrender.com/profissional`

`https://sprint-04-ddd.onrender.com/instituicao`

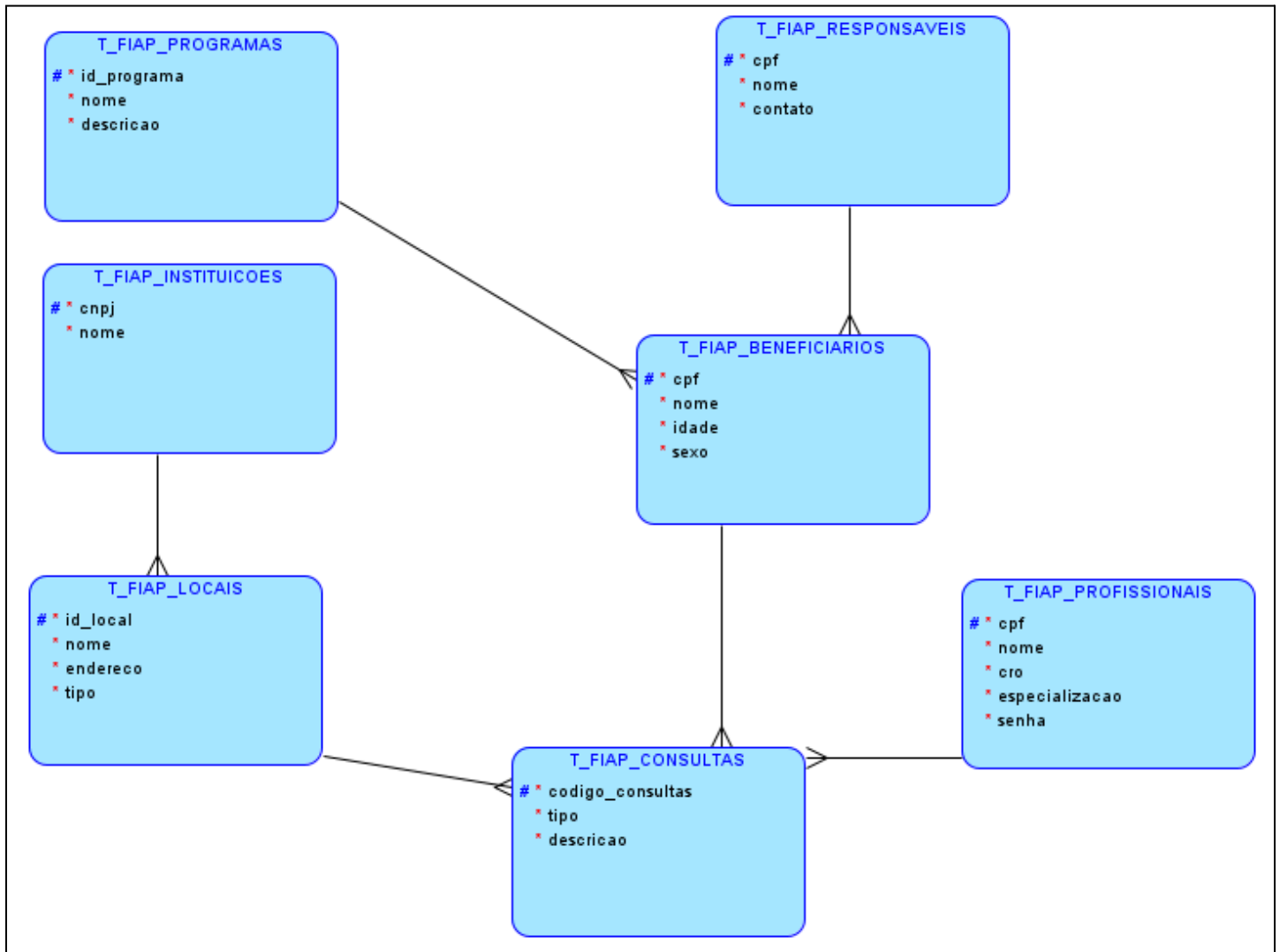
- Tecnologias Utilizadas

- Java 17+
- Quarkus 3.35.4
- Oracle JDBC (ojdbc11 23.3.0)
- Banco de Dados Oracle FIAP
- Jakarta REST (JAX-RS)
- Maven

Diagrama de Classes



Modelo de Entidade-Relacionamento





Login

Acesse o painel com seu CRO e senha

Entrar

[Não tem conta? Cadastre-se](#)

Esta tela representa a funcionalidade de autenticação da aplicação, permitindo que profissionais cadastrados realizem acesso ao sistema por meio da integração com a API REST desenvolvida no backend. O componente foi implementado utilizando React com TypeScript, React Hook Form para gerenciamento e validação dos campos do formulário e React Router para controle da navegação entre páginas.

O formulário solicita o CRO e a senha do profissional, realizando validações obrigatórias antes do envio dos dados. Após a submissão, a aplicação envia uma requisição HTTP do tipo POST para o endpoint `/profissional/login` da API REST, utilizando o método `fetch` e o formato JSON para transmissão das informações.

Quando a autenticação é concluída com sucesso, os dados retornados pela API são armazenados no `localStorage` do navegador, permitindo manter as informações do profissional durante a sessão. Em seguida, o usuário é redirecionado automaticamente para a tela de dashboard da aplicação.

Caso as credenciais estejam incorretas ou ocorra falha de comunicação com o servidor, mensagens de erro são exibidas dinamicamente na interface, proporcionando melhor feedback ao usuário.

Cadastro de Profissional

Crie sua conta para acessar o painel

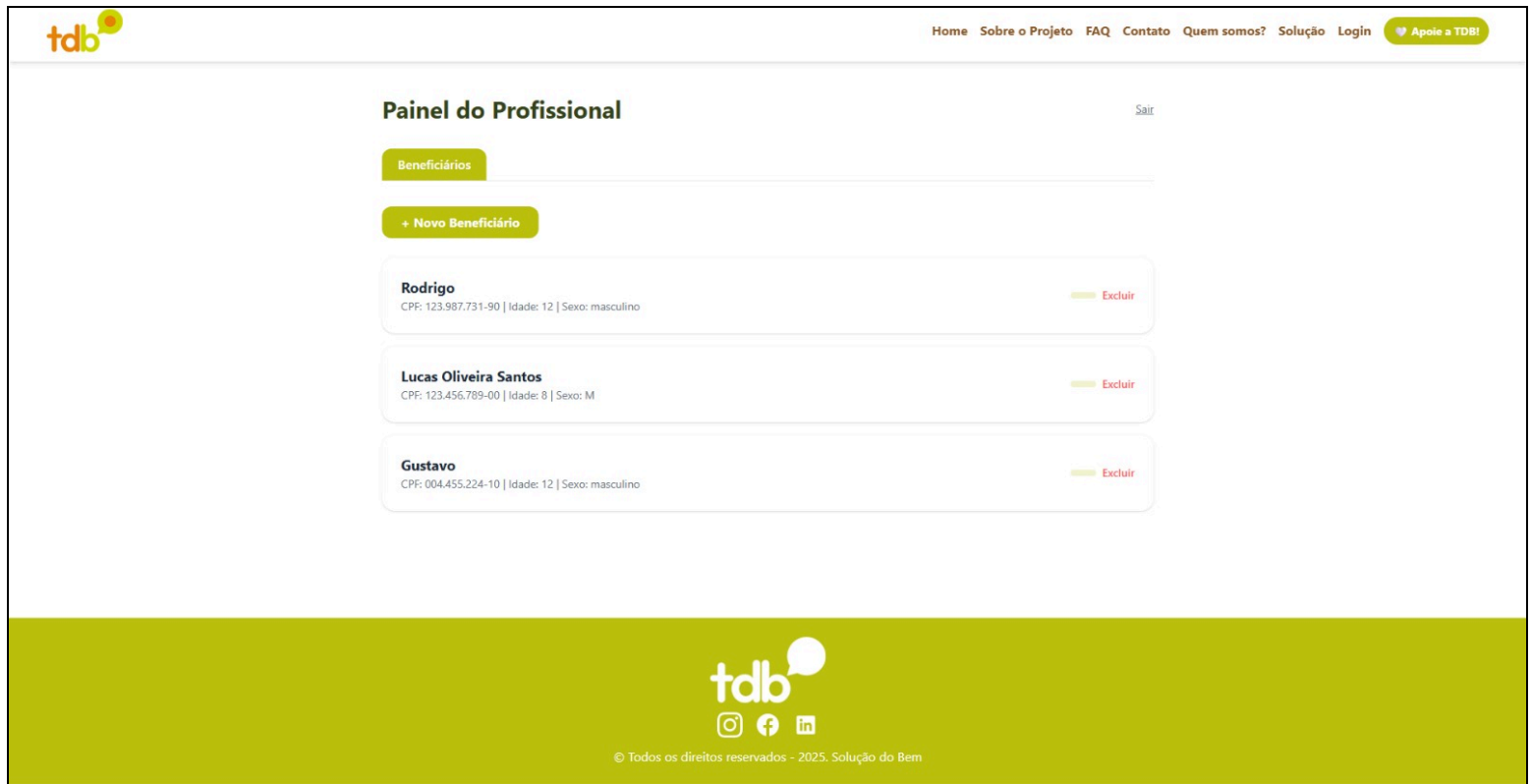
Cadastrar

[Já tem conta? Faça login](#)

Esta tela representa a funcionalidade de cadastro de profissionais da plataforma, permitindo que dentistas realizem seu registro para obter acesso ao sistema. O componente foi desenvolvido utilizando React com TypeScript, React Hook Form para gerenciamento e validação dos formulários, além de React Router para controle da navegação entre páginas.

O formulário solicita informações essenciais do profissional, como nome completo, CPF, CRO, especialização e senha. Todos os campos possuem validação obrigatória, garantindo maior integridade dos dados inseridos pelo usuário. Após o envio do formulário, as informações são enviadas para a API REST da aplicação por meio de uma requisição HTTP do tipo POST.

Quando o cadastro é realizado com sucesso, uma mensagem de confirmação é exibida na interface e o usuário é redirecionado automaticamente para a tela de login. Caso ocorra alguma falha durante o processo, mensagens de erro são apresentadas para orientar o usuário.



Esta tela representa o painel principal do profissional dentro da plataforma, permitindo o gerenciamento de beneficiários e consultas de forma centralizada e organizada. O componente foi desenvolvido utilizando também React com TypeScript, React Hook Form para gerenciamento de formulários e React Router para navegação entre páginas.

A interface possui um sistema de abas que separa as funcionalidades da aplicação em diferentes seções, facilitando a navegação e a visualização das informações. Na aba de beneficiários, o profissional pode cadastrar novos pacientes, informando dados pessoais, programa de atendimento e informações do responsável. Após o envio do formulário, os dados são enviados para a API REST da aplicação e armazenados no sistema. Também é possível visualizar e excluir beneficiários cadastrados.

Na aba de consultas, são exibidas as consultas registradas no sistema, contendo informações como beneficiário, tipo de consulta, descrição, local e código identificador. Os dados são carregados dinamicamente a partir da API utilizando o hook `useEffect`.